

Interação com APIs

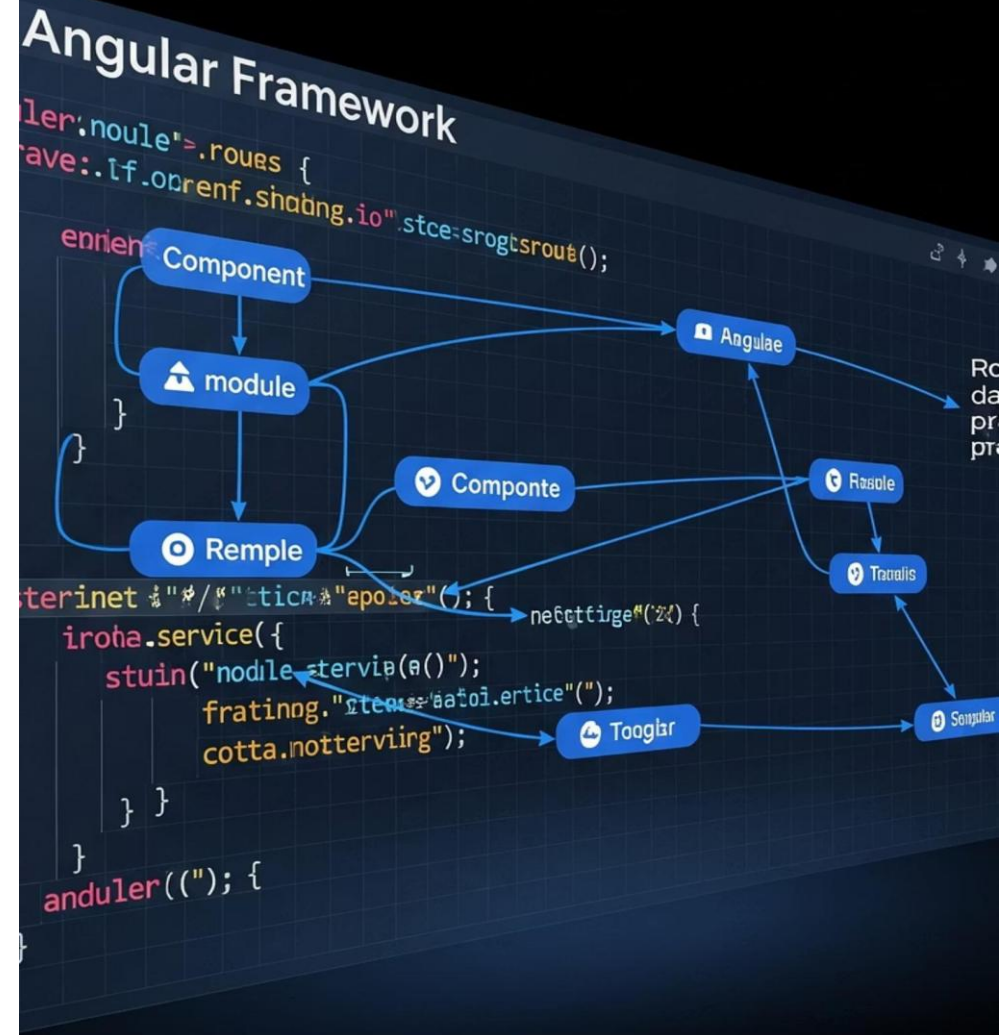
04

2026

SENAI - CRTI

Consumo de APIs no Angular

Nesta aula, vamos realizar nossa primeira integração real com uma API pública usando o Angular. Você aprenderá a configurar o **HttpClient**, criar serviços reutilizáveis, entender o **assincronismo** e diferenciar **Promise**, **Async/Await** e **Observable** — fundamentos essenciais para qualquer desenvolvedor front-end no mercado de trabalho.



O Desafio da Software House

Contexto profissional

Uma software house recebeu a demanda de construir um **sistema corporativo de consulta de endereços**. Ao digitar um CEP válido, a aplicação deve buscar automaticamente: rua, bairro, cidade e estado — em tempo real, sem recarregar a página.



Pergunta-problema: **Como buscar dados externos em tempo real e renderizá-los dinamicamente na interface?**

O que vamos construir

- Usuário digita o CEP
Input na interface Angular
- Angular chama a API ViaCEP
Usando HttpClient + Service
- Dados aparecem na tela
Rua, bairro, cidade, estado

Por Que Consumir APIs?

Aplicações Angular normalmente existem para **consumir serviços externos**. Quase toda tela de uma aplicação real depende de dados vindos de um servidor ou de terceiros.



Consulta de Endereço

CEP → rua, bairro, cidade via ViaCEP



Catálogo de Produtos

Listagem dinâmica de itens



Dados Meteorológicos

Temperatura, previsão do tempo



Dashboard

Métricas e indicadores em tempo real



Consulta de Pedidos

Status e histórico de compras



Dados Bancários

Saldo, extrato, transferências

O que é o HttpClient?

Definição

O **HttpClient** é o serviço oficial do Angular para comunicação HTTP. Ele permite que sua aplicação front-end se comunique com servidores back-end e APIs externas de forma simples, tipada e reativa.

Faz parte do módulo `@angular/common/http` e é a forma recomendada pelo próprio Angular para qualquer operação de rede.

Operações suportadas

GET

Buscar dados

POST

Enviar dados

PUT

Atualizar tudo

PATCH

Atualizar parcial

DELETE

Remover dados

Recursos avançados

- **Headers** customizados para autenticação e controle
- **Interceptors** para processar todas as requisições
- **Tratamento de erros** centralizado e robusto

Habilitando o HttpClient na Aplicação

Antes de usar o **HttpClient**, é necessário registrá-lo nos providers da aplicação. Com o Angular moderno (standalone), isso é feito no arquivo `app.config.ts`:

```
import { ApplicationConfig,
  provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient()
  ]
};
```

Por que é necessário?

Injeção de dependência

O Angular precisa saber que o HttpClient está disponível para ser injetado nos services e componentes.

Habilita recursos HTTP

Ativa interceptors, tipagem de resposta, gerenciamento de requisições e todas as funcionalidades HTTP da aplicação.

✓ Configure uma única vez e o HttpClient estará disponível em toda a aplicação via injeção de dependência.

Criando o Model de CEP (models/cep.ts)

Vamos criar o modelo de um cep, como a API define por contrato:

```
export interface Cep {  
  cep: string;  
  logradouro: string;  
  complemento: string;  
  bairro: string;  
  localidade: string;  
  uf: string;  
  estado: string;  
  erro?: boolean;  
}
```

Criando o Service de CEP (services/cep.ts)

No Angular, a lógica de comunicação com APIs deve ser isolada em um **Service**. Isso garante separação de responsabilidades, reutilização e facilidade de testes. Veja a estrutura básica do nosso `CepService`:

```
import { inject, Service } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Cep } from '../models/cep';

@Service()
export class CepService {

  private http = inject(HttpClient);
```

@Service

Marca a classe como um serviço e a disponibiliza automaticamente no injetor raiz da aplicação — sem precisar declarar `providedIn` manualmente.

Separação de Camadas

O componente cuida da interface. O service cuida da comunicação. Essa divisão torna o código mais organizado e fácil de manter.

Reutilização

Um único service pode ser injetado em múltiplos componentes, evitando duplicação de código em toda a aplicação.

Implementando o Método GET

Código do método

```
buscarCep(cep: string) {  
    return this.http.get<Cep>(  
        `https://viacep.com.br/ws/${cep}/json/`  
    );  
}
```

 Este método deve ser adicionado dentro da classe CepService, em service

Entendendo cada parte

01

Parâmetro cep: string

Recebe o CEP digitado pelo usuário, tornando o endpoint dinâmico e reutilizável.

02

Template literal `\${cep}`

Interpola o CEP diretamente na URL da requisição usando crase e `\${}`.

03

Retorno assíncrono

O método retorna um Observable<Cep> — a resposta chegará no futuro, já tipada.

Criando o componente principal (app.ts)

```
import { Component, inject, signal } from '@angular/core';
import { FormControl, ReactiveFormsModule, Validators } from '@angular/forms';
import { CepService } from '../services/cep';
import { Cep } from '../models/cep';

@Component({
  selector: 'app-root',
  imports: [ReactiveFormsModule],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  private cepService = inject(CepService);

  cepControl = new FormControl('', [
    Validators.required,
    Validators.pattern(/^\d{8}$/)
  ]);
```

Criando o componente principal (app.ts)

```
endereco = signal<Cep | null>(null);
loading = signal(false);
erro = signal('');

buscar() {
  if (this.cepControl.invalid) {
    this.erro.set('Digite um CEP válido com 8 números.');
```



```
    return;
  }

  this.loading.set(true);
  this.erro.set('');
  this.endereco.set(null);

  const cep = this.cepControl.value!;
```

Criando o componente principal (app.ts)

```
this.cepService.buscarCep(cep).subscribe({
  next: (resposta) => {
    this.loading.set(false);

    if (resposta.erro) {
      this.erro.set('CEP não encontrado. ');
      return;
    }

    this.endereco.set(resposta);
  },
  error: () => {
    this.loading.set(false);
    this.erro.set('Erro ao consultar API. ');
  }
});
}
```

Criando o HTML (app.html)

```
<div class="container">
  <h1>Consulta CEP</h1>

  <div class="busca">
    <input
      type="text"
      placeholder="Digite o CEP (somente números)"
      [formControl]="cepControl"
    />
    <button (click)="buscar()">
      Buscar
    </button>
  </div>
```

Criando o HTML (app.html)

```
@if (loading()) {  
  <p class="loading">Consultando...</p>  
}  
@if (erro()) {  
  <p class="erro">{{ erro() }}</p>  
}  
@if (endereco(); as cep) {  
  <div class="card">  
    <h2>Endereço encontrado</h2>  
    <p><strong>CEP:</strong> {{ cep.cep }}</p>  
    <p><strong>Rua:</strong> {{ cep.logradouro }}</p>  
    <p><strong>Bairro:</strong> {{ cep.bairro }}</p>  
    <p><strong>Cidade:</strong> {{ cep.localidade }}</p>  
    <p><strong>UF:</strong> {{ cep.uf }}</p>  
  </div>  
}  
</div>
```

Criando o CSS (app.css)

```
* {  
  box-sizing: border-box;  
  margin: 0;  
  padding: 0;  
  font-family: Arial, sans-serif;  
}  
body {  
  background: #f5f7fb;  
}  
.container {  
  width: 100%;  
  max-width: 700px;  
  margin: 60px auto;  
  padding: 30px;  
}  
h1 {  
  text-align: center;  
  margin-bottom: 30px;  
  color: #222;  
}
```

Criando o CSS (app.css)

```
.busca {  
  display: flex;  
  gap: 12px;  
  margin-bottom: 25px;  
}  
input {  
  flex: 1;  
  padding: 14px;  
  border: 1px solid #ccc;  
  border-radius: 8px;  
  font-size: 16px;  
}  
button {  
  border: none;  
  background: #2563eb;  
  color: white;  
  padding: 14px 24px;  
  border-radius: 8px;  
  cursor: pointer;  
  font-weight: bold;  
}
```

Criando o CSS (app.css)

```
button:hover {  
  background: #1d4ed8;  
}  
.card {  
  background: white;  
  padding: 25px;  
  border-radius: 12px;  
  box-shadow: 0 4px 18px rgba(0,0,0,.08);  
}  
.card h2 {  
  margin-bottom: 15px;  
  color: #111;  
}  
.card p {  
  margin: 8px 0;  
  color: #444;  
}
```

Criando o CSS (app.css)

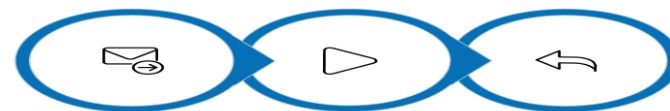
```
.loading {  
  color: #2563eb;  
  margin-bottom: 20px;  
  font-weight: bold;  
}  
.erro {  
  color: #dc2626;  
  margin-bottom: 20px;  
  font-weight: bold;  
}
```

O que é Assincronismo?

Em aplicações web, o assincronismo permite que o navegador execute outras tarefas enquanto espera uma resposta do servidor. A aplicação não "trava" — ela continua respondendo ao usuário.

Analogia: É como pedir comida em um restaurante. Você faz o pedido (request), continua conversando (a aplicação segue funcionando) e o garçom traz a comida quando fica pronta (callback com a resposta).

No Angular, toda operação HTTP é assíncrona por padrão. Por isso precisamos de mecanismos como Promises, Async/Await e Observables para lidar com respostas que chegam no futuro.



Pedido enviado

App continua

Resposta &
callback

Promise e Async/Await

Exemplo com Async/Await e Fetch API - Service

```
buscarCep(cep: string) {  
  return firstValueFrom(  
    this.http.get<Cep>(  
      `https://viacep.com.br/ws/${cep}/json/`  
    )  
  );  
}
```

Componente

```
async buscar() {  
  try {  
    this.loading.set(true);  
    const resposta = await  
      this.cepService.buscarCep('01001000');  
    this.endereco.set(resposta);  
  } catch {  
    this.erro.set('Erro');  
  } finally {  
    this.loading.set(false);  
  }  
}
```

Como funciona?

01

Palavra-chave async

Declara que a função é assíncrona. Permite usar await dentro dela.

02

Palavra-chave await

Pausa a execução da função até a Promise ser resolvida, sem bloquear o resto da aplicação.

03

Código linear

Async/Await faz código assíncrono parecer síncrono, muito mais fácil de ler e manter do que callbacks aninhados.

Observable: O Padrão Angular para Fluxos Assíncronos

Observable é o padrão Angular para fluxos assíncronos. Fornecido pela biblioteca **RxJS**, o Observable é muito mais poderoso que uma Promise — ele pode emitir múltiplos valores ao longo do tempo, ser cancelado e transformado com operadores.



Stream de Dados

Emite zero, um ou múltiplos valores ao longo do tempo, ao contrário da Promise que resolve uma única vez.



Subscribe

Para receber os dados, você se inscreve no Observable. Sem subscribe, nada acontece.



Operators

Funções como map, filter, debounceTime transformam e combinam fluxos de dados.



Cancelamento

Diferente de Promises, Observables podem ser cancelados com unsubscribe(), liberando recursos.



Composição

Múltiplos fluxos podem ser combinados, encadeados e transformados para criar lógicas complexas com elegância.

Promise vs Async/Await vs Observable

Critério	Promise	Async/Await	Observable
Valores emitidos	Um único valor	Um único valor	Múltiplos valores
Cancelável?	Não	Não	Sim (unsubscribe)
Operadores	Básico (.then)	Básico (await)	Poderosos (RxJS)
Legibilidade	Média	Alta (linear)	Alta (com prática)
Padrão Angular	Compatível	Compatível	Nativo ✓
Ideal para	Operações simples	Código legível	Fluxos reativos

✓ **No Angular, prefira Observables com subscribe()** ao usar o HttpClient. É o padrão da plataforma e habilita recursos avançados do RxJS.

A API ViaCEP em Ação

Endpoint que vamos usar

```
https://viacep.com.br/ws/20040002/json/
```

Resposta JSON da API

```
{
  "cep": "20040-002",
  "logradouro": "Avenida Rio Branco",
  "bairro": "Centro",
  "localidade": "Rio de Janeiro",
  "uf": "RJ"
}
```

Fluxo completo da integração

Usuário digita CEP

Service chama HttpClient

GET para ViaCEP

JSON retornado



A API ViaCEP é pública, gratuita e não requer autenticação — perfeita para aprender consumo de APIs.

Debate: Decisões de Arquitetura

Bons desenvolvedores não apenas escrevem código que funciona — eles pensam nas consequências das decisões que tomam. Reflita sobre as questões abaixo:

Component deve chamar a API diretamente?

Qual o problema de colocar o `HttpClient` diretamente no componente? O que acontece quando a URL da API muda?

Promise ou Observable?

Em quais situações você escolheria um ou outro? Pense em um chat em tempo real — qual seria mais adequado?

Como lidar com lentidão na rede?

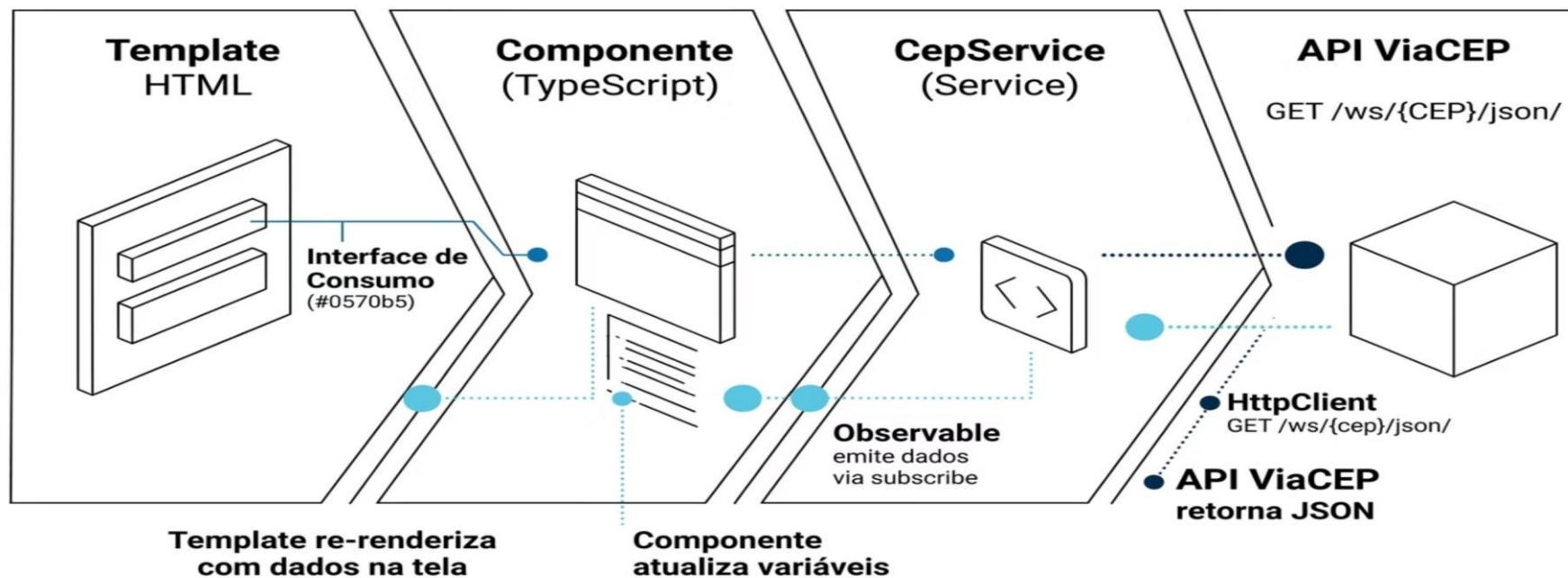
Se a API demorar 10 segundos para responder, o que acontece na interface? Como melhorar a experiência do usuário?

O que acontece com erros HTTP?

Se o CEP não existir, a ViaCEP retorna um JSON com `erro: true`. Como você vai tratar isso no subscribe?

Fluxo Visual Completo da Aula

Veja como todas as peças se encaixam em uma aplicação Angular que consome APIs:



Este fluxo representa a base de qualquer aplicação Angular que consome dados externos. Dominar cada etapa é dominar o desenvolvimento front-end moderno.

Próximos passos

Expandindo o Conhecimento

O que você pode explorar a seguir

01

Tratamento de erros com catchError

Operador RxJS para capturar e tratar falhas nas requisições HTTP.

02

Tipagem da resposta com interfaces

Criar uma interface Cep para tipar o retorno da API e ter autocomplete no TypeScript.

03

Interceptors HTTP

Adicionar tokens de autenticação automaticamente em todas as requisições.

04

Operadores RxJS avançados

debounceTime, switchMap — buscar enquanto digita, sem sobrecarregar a API.

Recursos de estudo recomendados

Documentação Angular

angular.dev/guide/http — guia oficial do HttpClient com exemplos detalhados.

API ViaCEP

viacep.com.br — documentação da API pública que usamos nesta aula.

RxJS Docs

rxjs.dev — referência completa de Observables e operadores reativos.

O Que Você é Capaz de Fazer Agora

Ao concluir esta aula com a atividade guiada, você demonstrou as seguintes competências profissionais, alinhadas ao perfil de desenvolvedor Angular no mercado:

✓ **Configurar HttpClient**

Habilitar e registrar o HttpClient na aplicação Angular com `provideHttpClient()`.

✓ **Criar Services**

Estruturar a camada de serviços com separação de responsabilidades e injeção de dependência.

✓ **Consumir GET**

Realizar requisições HTTP GET com endpoint dinâmico via template literal.

✓ **Entender subscribe**

Ativar Observables e processar respostas assíncronas no componente.

✓ **Diferenciar Promise / Async / Observable**

Compreender cada modelo assíncrono e aplicar o mais adequado para cada situação.

✓ **Integrar Angular com API real**

Conectar a interface front-end à API ViaCEP e renderizar dados externos dinamicamente.

✓ **Parabéns!** Você realizou sua primeira integração real com uma API em Angular. Esta é a base de praticamente todas as aplicações web modernas.

Firjan SENAI

